ASSIGNMENT-16

HALLTICKET:2303A510F7

BATCH:03

LAB – 16 : Database Design and Queries : Schema Design and SQL Generation.

Task – 01 : Design Database Schema for Student Management System.

Prompt : Generate SQL CREATE TABLE statements for a Student Management System with tables Students, Courses, and Enrollments. Include primary keys, foreign keys, proper data types, and ensure normalization up to 3NF.

Code :

```sql
--TASK 01--

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT CHECK (credits > 0)
);

CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
```

```sql
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
        ON DELETE CASCADE
);
```

Explanation :

The schema is normalized (1NF, 2NF, 3NF) by separating entities into different tables. Primary keys uniquely identify records, while foreign keys maintain relationships. Constraints like NOT NULL, UNIQUE, and CHECK ensure data integrity. ON DELETE CASCADE ensures related records are removed automatically

Output :

Courses

course_id	course_name	credits
empty		

Enrollments

enrollment_id	student_id	course_id
empty		

Students

student_id	name	email
empty		

Task – 02 : Insert Sample Records.

Prompt : Generate SQL INSERT statements to populate Students, Courses, and Enrollments tables with at least 3 students, 4 courses, and 5 enrollments.

Code :

```sql
--TASK 02--
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);

SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

OUTPUT :

Courses

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

Enrollments

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101
5	3	104

Students

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

Explanation :

Sample data is inserted into all tables to simulate a real system. Each enrollment links a student to a course using foreign keys. SELECT queries help verify successful insertion of records. Ensures relational consistency across tables.

Task – 03 : Perform CURD Operations.

Prompt : Generate SQL queries to perform CRUD operations: retrieve all courses, update a student's email, and delete an enrollment record.

Code :

```sql
--TASK 03--

SELECT * FROM Courses;

UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;

SELECT * FROM Students WHERE student_id = 1;

DELETE FROM Enrollments
WHERE enrollment_id = 5;
SELECT * FROM Enrollments;
```

Output :

Output

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

student_id	name	email
1	Arjun	arjun_new@gmail.com

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Explanation :

SELECT retrieves data from the database. UPDATE modifies existing records based on conditions. DELETE removes specific records safely using a WHERE clause. Verification queries ensure changes are correctly applied.

Task – 04 : Execute Aggregate Queries.

Prompt : Generate SQL queries to count number of students per course and display courses with more than one student enrolled.

Code :

```sql
--TASK 04--

SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Output :

Output

course_name	student_count
Data Structures	1
Database Systems	2
Operating Systems	1

course_name
Database Systems

Explanation :

JOIN combines data from multiple tables based on relationships. COUNT() calculates total students in each course. GROUP BY groups results for aggregation. HAVING filters grouped results based on conditions.

Task – 05 : Query Optimization.

Prompt : Rewrite a subquery using JOIN for better performance and compare both queries.

Code :

```sql
--TASK 05--
-- Original Query (Subquery)
SELECT * FROM Students
WHERE student_id IN (
    SELECT student_id FROM Enrollments WHERE course_id = 101
);

-- Optimized Query (JOIN)
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Output :

Output

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

Explanation :

The original query uses a subquery, which can be slower for large datasets. The optimized query uses JOIN, improving performance and readability.
Both queries return the same results (students in course 101). JOIN-based queries are preferred in real-world applications.